

Advanced RAG Techniques

Lab Guide

Hands-On Python Implementation on AWS EC2 Ubuntu

LangChain

OpenAI API

SentenceTransformers

Docling

NetworkX

#	Module	Key Libraries	Difficulty
01	Reranking Retrieved Chunks	SentenceTransformers, LangChain	Intermediate
02	Multi-Query Retrieval	LangChain MultiQueryRetriever, FAISS	Intermediate
03	Knowledge Graph RAG	NetworkX, LangChain, spaCy	Advanced
04	Multimodal RAG with Docling	Docling, LangChain, OpenAI Vision	Advanced

Prerequisites

AWS EC2 Ubuntu 22.04 instance (t3.large or better) | Code-Server installed | Python 3.10+

OpenAI API Key | Internet access for pip installs | 8 GB RAM minimum

Table of Contents

Quick Navigation

Environment Setup & Project Structure	3
Module 01 – Reranking Retrieved Chunks	5
1.1 Concept Overview	5
1.2 Install Dependencies	5
1.3 Sample Insurance Documents	5
1.4 Build Vector Store (FAISS)	6
1.5 Cross-Encoder Reranker	6
1.6 Full Pipeline Code	7
1.7 Run & Observe Results	8
Module 02 – Multi-Query Retrieval	9
2.1 Concept Overview	9
2.2 Install Dependencies	9
2.3 MultiQueryRetriever Setup	10
2.4 Compare Results	11
Module 03 – Knowledge Graph RAG	12
3.1 Concept Overview	12
3.2 Install Dependencies	13
3.3 Build Entity Graph	13
3.4 Graph-Augmented RAG Pipeline	14
3.5 Query & Visualize	15
Module 04 – Multimodal RAG with Docling	16
4.1 Concept Overview	16
4.2 Install Dependencies	16
4.3 Ingest PDF + Image via Docling	17
4.4 Build RAG Pipeline	18
4.5 End-to-End Query	19
Troubleshooting & Common Errors	20
Expected Outputs Summary	21

Environment Setup & Project Structure

Configure your EC2 workspace before starting any module

1. Open Code-Server on your EC2 instance and open the integrated terminal.

1 Create & activate a virtual environment

```
cd ~  
mkdir rag_labs && cd rag_labs  
python3 -m venv venv  
source venv/bin/activate
```

2 Install common base packages

```
pip install --upgrade pip  
pip install openai langchain langchain-openai langchain-community  
pip install faiss-cpu tiktoken python-dotenv
```

3 Configure OpenAI API key

```
# Create .env file in rag_labs/  
echo 'OPENAI_API_KEY=sk-YOUR_KEY_HERE' > .env
```

4 Recommended project structure

```
rag_labs/  
├── .env # API keys  
├── venv/ # virtual environment  
├── module01_reranking/  
│   └── rerank_rag.py  
├── module02_multiquery/  
│   └── multiquery_rag.py  
├── module03_graph/  
│   └── graph_rag.py  
├── module04_docling/  
│   └── docling_rag.py  
├── sample_policy.pdf  
├── claim_image.png  
├── data/  
└── insurance_docs.txt
```

Note

Each module is self-contained. Activate the venv before running any script: `source ~/rag_labs/venv/bin/activate`

Module 01 — Reranking Retrieved Chunks

Use Cross-Encoder scores to boost answer quality | Libraries: SentenceTransformers, LangChain, FAISS

Learning Outcomes

- ✓ Understand why bi-encoder retrieval alone can return irrelevant chunks
- ✓ Apply a Cross-Encoder reranker to re-score and filter retrieved documents
- ✓ Compare answer quality before and after reranking in an insurance Q&A; scenario

1.1 Concept Overview

Term	Description
Bi-Encoder	Converts query & documents to embeddings independently. Fast, scalable, but scores not perfectly calibrated.
Cross-Encoder	Takes query + document TOGETHER as input, producing a single relevance score. Slower but highly accurate.
Reranking Pipeline	Retrieve top-K with bi-encoder, then rerank with cross-encoder, keep top-N. Best of both worlds.

Architecture Flow



1.2 Install Dependencies

1 Install SentenceTransformers and supporting packages

```
pip install sentence-transformers
```

1.3 Sample Insurance Documents (data/insurance_docs.txt)

Create this file with representative domain content:

```
# Save as: rag_labs/data/insurance_docs.txt

Policy P-1001: Comprehensive health cover for family plans.
Premium is Rs.18000/year. Covers hospitalization, day-care, and
pre-existing diseases after 2-year waiting period.

Claim C-2045: Cashless hospitalization request by customer
Ravi Kumar at Apollo Hospital Chennai for cardiac procedure.
Approved amount: Rs.2,50,000. Status: Approved.

Policy P-1002: Motor vehicle insurance. Covers own-damage,
third-party liability. No-claim bonus of 20% after 3 claim-free years.
```

```
Claim C-2046: Vehicle accident claim by Priya Sharma.  
Damages estimated at Rs.45,000. Surveyor assessment pending.  
Deductible: Rs.5,000.  
  
Agent A-301: Senior relationship manager handling tier-1 customers.  
Renewal rate: 94%. Average portfolio: 120 policies.
```

1.4 Build Vector Store

2 Create module01_reranking/rerank_rag.py — Part 1: Vector Store

```
# module01_reranking/rerank_rag.py  
  
import os  
from dotenv import load_dotenv  
from langchain_openai import OpenAIEmbeddings, ChatOpenAI  
from langchain_community.vectorstores import FAISS  
from langchain.text_splitter import RecursiveCharacterTextSplitter  
from langchain.schema import Document  
  
load_dotenv()  
  
# --- Load documents ---  
with open('../data/insurance_docs.txt', 'r') as f:  
    raw_text = f.read()  
  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=200, chunk_overlap=30  
)  
chunks = splitter.create_documents([raw_text])  
print(f'Created {len(chunks)} chunks')  
  
# --- Build FAISS vector store ---  
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')  
vectorstore = FAISS.from_documents(chunks, embeddings)  
retriever = vectorstore.as_retriever(search_kwargs={'k': 8})  
print('Vector store built successfully.')
```

1.5 Cross-Encoder Reranker

3 Add reranking logic to the same file — Part 2

```
from sentence_transformers import CrossEncoder  
  
# --- Load Cross-Encoder model ---  
# Model will be downloaded on first run (~90 MB)  
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
```

```
def rerank_documents(query, docs, top_n=3):
    """Score query-document pairs and return top-N."""
    pairs = [(query, doc.page_content) for doc in docs]
    scores = reranker.predict(pairs)
    scored = list(zip(scores, docs))
    scored.sort(key=lambda x: x[0], reverse=True)
    print('\n--- Reranking Scores ---')
    for score, doc in scored:
        snippet = doc.page_content[:60].replace('\n', ' ')
        print(f' {score:.4f} | {snippet}...')
    return [doc for _, doc in scored[:top_n]]
```

1.6 Full Pipeline Code

4 Add the RAG pipeline — Part 3

```
from langchain.chains import RetrievalQA
from langchain.schema.runnable import RunnableLambda
from langchain.prompts import PromptTemplate

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

PROMPT = PromptTemplate(
    input_variables=['context', 'question'],
    template='''
You are an insurance assistant. Use the context below to answer.
Context: {context}
Question: {question}
Answer concisely with specific details from the context.'''
)

def rag_with_reranking(query):
    # Step 1: Retrieve top-8 candidates
    raw_docs = retriever.invoke(query)
    print(f'Retrieved {len(raw_docs)} docs before reranking')

    # Step 2: Rerank, keep top-3
    reranked = rerank_documents(query, raw_docs, top_n=3)
    context = '\n\n'.join([d.page_content for d in reranked])

    # Step 3: Generate answer
    response = llm.invoke(PROMPT.format(
        context=context, question=query
    ))
    return response.content
```

```
def rag_without_reranking(query):
    raw_docs = retriever.invoke(query)[:3]
    context = '\n\n'.join([d.page_content for d in raw_docs])
    response = llm.invoke(PROMPT.format(
        context=context, question=query
    ))
    return response.content
```

1.7 Run & Observe Results

5 Add main block and run the file

```
if __name__ == '__main__':
    query = 'What is the approved amount for Ravi Kumar claim?'

    print('\n===== WITHOUT Reranking =====')
    print(rag_without_reranking(query))

    print('\n===== WITH Reranking =====')
    print(rag_with_reranking(query))
```

```
# Run from rag_labs/module01_reranking/

python rerank_rag.py
```

Expected Console Output (excerpt)

```
Retrieved 8 docs before reranking
--- Reranking Scores ---
 9.2341 | Claim C-2045: Cashless hospitalization request by customer Ravi...
 4.1203 | Approved amount: Rs.2,50,000. Status: Approved...
 1.9871 | Policy P-1001: Comprehensive health cover for family...
WITH Reranking: The approved amount for Ravi Kumar's claim (C-2045) is Rs.2,50,000.
```


Module 02 — Multi-Query Retrieval

Generate query variations → retrieve broader context | Libraries: LangChain MultiQueryRetriever, FAISS, OpenAI

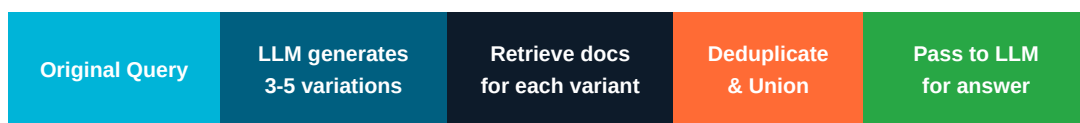
Learning Outcomes

- ✓ Understand query ambiguity and the limitation of single-query retrieval
- ✓ Use LangChain MultiQueryRetriever to automatically generate 3-5 query variations
- ✓ Observe how multiple query perspectives retrieve a more complete set of documents
- ✓ Compare single-query vs multi-query answers side by side

2.1 Concept Overview

Term	Description
Query Expansion	Rewriting the user query in multiple ways to increase recall from the vector store.
MultiQueryRetriever	LangChain component that uses an LLM to generate N alternative queries, retrieves documents for each, then deduplicates.
Union Strategy	Results from all sub-queries are combined with deduplication to maximise coverage.

How MultiQueryRetriever Works



2.2 Install Dependencies

```
# Already installed in base setup

# Verify with:

python -c 'from langchain.retrievers.multi_query import MultiQueryRetriever; print("OK")'
```

2.3 MultiQueryRetriever Setup

1 Create module02_multiquery/multiquery_rag.py

```
# module02_multiquery/multiquery_rag.py

import os, logging
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain.chains import RetrievalQA
```

```

load_dotenv()

logging.basicConfig()

# Enable verbose to see generated sub-queries
logging.getLogger('langchain.retrievers.multi_query').setLevel(logging.INFO)

# --- Load and chunk documents ---
with open('../data/insurance_docs.txt', 'r') as f:
    raw_text = f.read()

splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=30)
chunks = splitter.create_documents([raw_text])

embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = FAISS.from_documents(chunks, embeddings)
base_retriever = vectorstore.as_retriever(search_kwargs={'k': 3})

```

2 Initialise MultiQueryRetriever

```

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

# MultiQueryRetriever wraps the base retriever
# It will ask the LLM to generate query variants
multi_retriever = MultiQueryRetriever.from_llm(
    retriever=base_retriever,
    llm=llm
)

```

3 Define single-query and multi-query RAG functions

```

from langchain.prompts import PromptTemplate

PROMPT = PromptTemplate(
    input_variables=['context', 'question'],
    template=''
    You are an insurance assistant. Use context below.
    Context: {context}
    Question: {question}
    Provide a specific and accurate answer.'''
)

def single_query_rag(query):
    docs = base_retriever.invoke(query)
    context = '\n\n'.join([d.page_content for d in docs])
    return llm.invoke(PROMPT.format(context=context, question=query)).content, len(docs)

def multi_query_rag(query):
    # MultiQueryRetriever auto-generates sub-queries

```

```
docs = multi_retriever.invoke(query) # returns deduplicated union
context = '\n\n'.join([d.page_content for d in docs])
return llm.invoke(PROMPT.format(context=context, question=query)).content, len(docs)
```

2.4 Compare Results

4 Add comparison runner and execute

```
if __name__ == '__main__':
    queries = [
        'What policies cover hospitalization?',
        'Tell me about motor insurance no-claim benefits',
        'What claims are pending surveyor review?'
    ]

    for q in queries:
        print(f'\nQUERY: {q}')
        print('-' * 60)
        s_ans, s_cnt = single_query_rag(q)
        m_ans, m_cnt = multi_query_rag(q)
        print(f'Single Query ({s_cnt} docs): {s_ans}')
        print(f'Multi Query ({m_cnt} docs): {m_ans}')
```

```
python multiquery_rag.py
```

Observation	Single Query	Multi Query
Docs Retrieved	3 (fixed)	6-12 (deduplicated union)
Query Coverage	Literal match only	Multiple angles covered
Answer Completeness	May miss related info	Broader, more complete
LLM Calls	1 (generation only)	2 (expansion + generation)

Module 03 — Knowledge Graph RAG

Build insurance entity graph → query via graph-augmented retrieval | Libraries: NetworkX, LangChain, spaCy

Learning Outcomes

- ✓ Model insurance domain entities (Customer, Policy, Claim, Agent) as a graph
- ✓ Traverse the graph to retrieve structurally related context
- ✓ Combine graph context with vector search for richer answers
- ✓ Understand when graph RAG outperforms pure vector RAG

3.1 Concept Overview

Term	Description
Knowledge Graph	A directed graph where nodes are entities and edges are typed relationships. Enables multi-hop traversal.
Graph RAG	At query time, identify relevant entities, traverse neighbours in the graph, and pass structured context to LLM.
Hybrid Retrieval	Combine graph traversal context (structured) + FAISS vector search (semantic) for best coverage.

Entity Graph Design

Node Type	Example	Relationships (→)
Customer	Ravi Kumar, Priya Sharma	HOLDS_POLICY, FILED_CLAIM
Policy	P-1001, P-1002	COVERS, MANAGED_BY
Claim	C-2045, C-2046	LINKED_TO_POLICY, REVIEWED_BY
Agent	Agent A-301	MANAGES_POLICY, SERVES_CUSTOMER

3.2 Install Dependencies

```
pip install networkx spacy
python -m spacy download en_core_web_sm
```

3.3 Build the Entity Graph

1 Create module03_graph/graph_rag.py — Build graph

```
# module03_graph/graph_rag.py
import os, networkx as nx
```

```

from dotenv import load_dotenv

from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter

load_dotenv()

# ---- Build Knowledge Graph ----
G = nx.DiGraph()

# Add nodes with metadata
G.add_node('Ravi Kumar', type='Customer', tier='Premium')
G.add_node('Priya Sharma', type='Customer', tier='Standard')
G.add_node('P-1001', type='Policy', product='Health', premium=18000)
G.add_node('P-1002', type='Policy', product='Motor', premium=12000)
G.add_node('C-2045', type='Claim', status='Approved', amount=250000)
G.add_node('C-2046', type='Claim', status='Pending', amount=45000)
G.add_node('Agent-301', type='Agent', renewal_rate=0.94)

# Add typed relationships
G.add_edge('Ravi Kumar', 'P-1001', relation='HOLDS_POLICY')
G.add_edge('Ravi Kumar', 'C-2045', relation='FILED_CLAIM')
G.add_edge('C-2045', 'P-1001', relation='LINKED_TO_POLICY')
G.add_edge('Priya Sharma', 'P-1002', relation='HOLDS_POLICY')
G.add_edge('Priya Sharma', 'C-2046', relation='FILED_CLAIM')
G.add_edge('C-2046', 'P-1002', relation='LINKED_TO_POLICY')
G.add_edge('Agent-301', 'P-1001', relation='MANAGES_POLICY')
G.add_edge('Agent-301', 'P-1002', relation='MANAGES_POLICY')

print(f'Graph: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges')

```

3.4 Graph-Augmented RAG Pipeline

2 Add graph traversal and vector store

```

# ---- Vector Store (same insurance docs) ----
with open('../data/insurance_docs.txt', 'r') as f:
    raw_text = f.read()

splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=30)
chunks = splitter.create_documents([raw_text])
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = FAISS.from_documents(chunks, embeddings)

# ---- Graph context retrieval ----
def get_graph_context(entity_name):

```

```

"""Return structured context by traversing graph neighbours."""
if entity_name not in G:
    return f'Entity {entity_name} not found in graph.'

node_data = G.nodes[entity_name]
ctx_lines = [f'Entity: {entity_name} | Type: {node_data.get("type","?")}']
ctx_lines.append(f'Attributes: {node_data}')

# Outgoing edges (what this entity relates to)
for _, neighbour, data in G.out_edges(entity_name, data=True):
    n_data = G.nodes[neighbour]
    ctx_lines.append(
        f' {data["relation"]} -> {neighbour} ({n_data})'
    )

# Incoming edges (what relates to this entity)
for source, _, data in G.in_edges(entity_name, data=True):
    s_data = G.nodes[source]
    ctx_lines.append(
        f' <- {source} [{data["relation"]} ({s_data})'
    )

return '\n'.join(ctx_lines)

```

3 Add hybrid query function

```

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)

def graph_rag_query(query, entity_hint=None):
    """Combine graph context + vector search for richer answers."""
    # 1. Graph context
    graph_ctx = ''
    if entity_hint and entity_hint in G:
        graph_ctx = get_graph_context(entity_hint)
        print(f'\n[Graph Context for {entity_hint}]\n{graph_ctx}')
    else:
        # Auto-detect entity from graph nodes
        for node in G.nodes():
            if node.lower() in query.lower():
                graph_ctx = get_graph_context(node)
                break

    # 2. Vector context
    vec_docs = vectorstore.similarity_search(query, k=3)
    vec_ctx = '\n'.join([d.page_content for d in vec_docs])

    # 3. Combined prompt

```

```
full_ctx = f'=== Graph Context ===\n{graph_ctx}\n\n=== Document Context ===\n{vec_ctx}'
prompt = f'Use the following context to answer: {full_ctx}\n\nQ: {query}'
return llm.invoke(prompt).content
```

3.5 Query & Visualize

4 Add queries and optional graph export

```
if __name__ == '__main__':
    questions = [
        ('What policy does Ravi Kumar hold and what claim did he file?',
         'Ravi Kumar'),
        ('What is the status of claim C-2046 and which policy is it linked to?',
         'C-2046'),
        ('How many policies does Agent-301 manage?',
         'Agent-301'),
    ]
    for q, entity in questions:
        print(f'\nQ: {q}')
        print(f'A: {graph_rag_query(q, entity_hint=entity)}')
        print('=' * 60)

    # Optional: Export graph as GEXF for Gephi
    nx.write_gexf(G, 'insurance_graph.gexf')
    print('\nGraph exported to insurance_graph.gexf')
```

```
python graph_rag.py
```

Note

The graph traversal returns structured relationship data (node attributes, edge types) that vector search cannot provide — e.g., the exact premium amount, renewal rate, or claim status. This structural context significantly improves answers for entity-centric questions.

Module 04 — Multimodal RAG with Docling

Ingest PDF + images via Docling → extract structured data → RAG pipeline | Libraries: Docling, LangChain, OpenAI

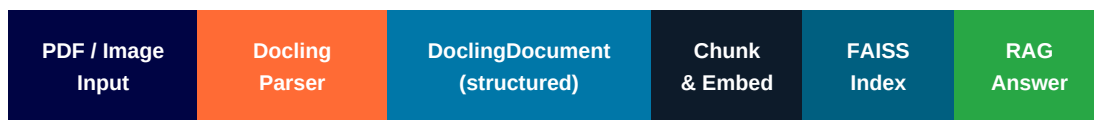
Learning Outcomes

- ✓ Use IBM Docling to parse PDFs and extract structured text, tables, and image captions
- ✓ Understand Docling's document model (DocItem types: TextItem, TableItem, PictureItem)
- ✓ Build a RAG pipeline over multimodal insurance documents
- ✓ Query across extracted text, tabular data, and image-derived content

4.1 Concept Overview

Term	Description
Docling	IBM open-source library that parses PDFs, DOCX, PPTX, images into a structured DoclingDocument with typed content items.
DocItem Types	TextItem (paragraphs), TableItem (structured tables → markdown/CSV), PictureItem (embedded images with captions).
Multimodal RAG	Index text, table summaries, and image captions together. LLM can then answer questions spanning all content types.

Docling Processing Pipeline



4.2 Install Dependencies

```

pip install docling

# Docling installs: easyocr, pytorch (CPU), transformers, etc.

# First run may take 5-10 min to download ML models.

# For image understanding via OpenAI Vision:
pip install Pillow
  
```

Note

Docling downloads layout and OCR models (~1-2 GB) on first use. Ensure your EC2 instance has at least 8 GB RAM and 10 GB free disk space. A t3.large (8 GB RAM) instance is the recommended minimum.

4.3 Ingest PDF + Image via Docling

1 Create a sample policy PDF for testing

```
# Run this ONCE to create sample_policy.pdf
```



```
# module04_docling/create_sample_pdf.py

from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib import colors

doc = SimpleDocTemplate('sample_policy.pdf', pagesize=A4)
styles = getSampleStyleSheet()
story = []

story.append(Paragraph('Insurance Policy Document P-3001', styles['Title']))
story.append(Paragraph('Customer: Arjun Mehta | DOB: 15-Mar-1985', styles['Normal']))
story.append(Paragraph('Policy Type: Comprehensive Health | Issued: 01-Jan-2024', styles['Normal']))

data = [
    ['Coverage Item', 'Limit (Rs)', 'Co-pay %'],
    ['In-patient Hospitalisation', '5,00,000', '10%'],
    ['Day-care Procedures', '2,00,000', '0%'],
    ['Pre-existing Diseases', '3,00,000', '20%'],
    ['Ambulance Cover', '5,000', '0%'],
]

t = Table(data)
t.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,0), colors.navy),
    ('TEXTCOLOR', (0,0), (-1,0), colors.white),
    ('GRID', (0,0), (-1,-1), 0.5, colors.grey),
]))

story.append(t)

story.append(Paragraph('Exclusions: Cosmetic surgery, war injuries, self-inflicted injuries.',
styles['Normal']))

doc.build(story)

print('sample_policy.pdf created.')
```

```
python create_sample_pdf.py
```

2 Create module04_docling/docling_rag.py — Docling ingestion

```
# module04_docling/docling_rag.py

import os

from pathlib import Path
from dotenv import load_dotenv
from docling.document_converter import DocumentConverter
from langchain.schema import Document
```

```

load_dotenv()

def ingest_with_docling(file_path):
    """
    Convert PDF or image to structured text using Docling.
    Returns a list of LangChain Document objects.
    """
    converter = DocumentConverter()
    result = converter.convert(str(file_path))
    doc = result.document

    extracted_docs = []

    # Extract all text items
    for item, level in doc.iterate_items():
        from docling.datamodel.base_models import ItemLabel
        content = ''
        item_type = 'text'

        # Text paragraphs
        if hasattr(item, 'text') and item.text:
            content = item.text.strip()
            item_type = 'text'

        # Tables: export as markdown
        if hasattr(item, 'export_to_markdown'):
            try:
                content = item.export_to_markdown()
                item_type = 'table'
            except Exception:
                pass

        if content:
            extracted_docs.append(Document(
                page_content=content,
                metadata={'source': str(file_path), 'type': item_type}
            ))

    print(f'Docling extracted {len(extracted_docs)} items from {file_path.name}')
    return extracted_docs

```

4.4 Build RAG Pipeline over Docling Output

3 Add vector store and RAG query function

```

from langchain_openai import OpenAIEmbeddings, ChatOpenAI

```

```

from langchain_community.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.prompts import PromptTemplate

def build_rag_from_files(file_paths):
    all_docs = []
    for fp in file_paths:
        path = Path(fp)
        if path.suffix.lower() in ['.pdf', '.png', '.jpg', '.jpeg', '.tiff']:
            docs = ingest_with_docling(path)
        else:
            print(f'Skipping unsupported format: {fp}')
            continue
        all_docs.extend(docs)

    # Split long items further
    splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=50)
    chunks = splitter.split_documents(all_docs)
    print(f'Total chunks for indexing: {len(chunks)}')

    embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
    vectorstore = FAISS.from_documents(chunks, embeddings)
    return vectorstore

PROMPT = PromptTemplate(
    input_variables=['context', 'question'],
    template='''
    You are an insurance document analyst.

    Use the extracted document context below to answer precisely.

    Context: {context}

    Question: {question}

    Provide specific values, amounts, or details where available.'''
)

def docling_rag_query(vectorstore, query, k=4):
    llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
    docs = vectorstore.similarity_search(query, k=k)
    print(f'\n[Retrieved {len(docs)} chunks]')
    for i, d in enumerate(docs):
        print(f' Chunk {i+1} [{d.metadata["type"]}] : {d.page_content[:80]}...')
    context = '\n\n'.join([d.page_content for d in docs])
    response = llm.invoke(PROMPT.format(context=context, question=query))
    return response.content

```

4.5 End-to-End Query

4 Add main block and run

```
if __name__ == '__main__':
    # List your PDF and image files
    files = [
        'sample_policy.pdf',
        # 'claim_image.png', # Uncomment if you have an image
    ]

    print('Building vector store from Docling-parsed documents...')
    vs = build_rag_from_files(files)

    questions = [
        'What is the in-patient hospitalisation limit for policy P-3001?',
        'What are the exclusions in this policy?',
        'What is the co-pay percentage for pre-existing diseases?',
        'Who is the policy holder and what is their date of birth?',
    ]

    for q in questions:
        print(f'\nQ: {q}')
        print(f'A: {docling_rag_query(vs, q)}')
        print('-' * 60)
```

```
cd ~/rag_labs/module04_docling
python create_sample_pdf.py # Create test PDF first
python docling_rag.py # Run the RAG pipeline
```

Query Type	Example Q	Expected Source
Table lookup	Hospitalisation limit?	TableItem extracted from PDF table
Text extraction	Exclusions?	TextItem paragraph below table
Mixed	Customer name + policy type?	Multiple TextItems from header

Troubleshooting & Common Errors

Error	Fix	Module
<code>ModuleNotFoundError: sentence_transformers</code>	<code>pip install sentence-transformers</code>	Module 01
<code>FAISS not found / ImportError</code>	<code>pip install faiss-cpu</code> (not <code>faiss-gpu</code> on CPU instances)	All modules
<code>AuthenticationError: OpenAI API Key</code>	Verify <code>.env</code> contains <code>OPENAI_API_KEY=sk-...</code> and <code>load_dotenv()</code> is called	All modules
<code>spacy model not found: en_core_web_sm</code>	<code>python -m spacy download en_core_web_sm</code>	Module 03
<code>Docling download timeout / slow first run</code>	First run downloads ~1-2 GB models. Run in a tmux session. Ensure EC2 has internet.	Module 04
<code>MemoryError during Docling processing</code>	Upgrade to <code>t3.large</code> (8 GB RAM) or larger. Docling loads ML models into RAM.	Module 04
<code>NetworkX KeyError: node not in graph</code>	Check <code>entity_hint</code> spelling — graph keys are case-sensitive (e.g., 'Ravi Kumar' not 'ravi kumar')	Module 03
<code>CrossEncoder model download fails</code>	Check internet access on EC2. The model <code>cross-encoder/ms-marco-MiniLM-L-6-v2</code> is ~90 MB.	Module 01
<code>langchain_community not found</code>	<code>pip install langchain-community</code> (separate package from langchain core)	All modules
<code>RateLimitError from OpenAI</code>	Add <code>time.sleep(1)</code> between API calls or reduce <code>k</code> in retrievers	All modules

Expected Outputs Summary

After completing all four modules, participants should observe the following outputs and behaviours:

Module	Key Output	What to Observe	Success Indicator
01 Reranking	Reranking scores printed to console + improved answer	Cross-encoder scores differ significantly Top-ranked chunk is most relevant	Answer references exact claim amount Rs.2,50,000
02 Multi-Query	3-5 LLM-generated query variants printed via logging	Multi-query retrieves more docs (6/12) vs single-query (3).	Answer more complete with better coverage of all policies
03 Graph RAG	Graph context block printed before each answer	Graph provides structured attributes (status, amount, rate) not in raw text	Agent-301 correctly shows 2 extended policies
04 Docling	Docling extraction log showing text/table item counts	TableItem chunks contain markdown table extracted from PDF.	Correct hospitalisation limit Rs.5,00,000 from table

Quick Commands Reference

```
# Navigate and run each module
cd ~/rag_labs
source venv/bin/activate

cd module01_reranking && python rerank_rag.py && cd ..
cd module02_multiquery && python multiquery_rag.py && cd ..
cd module03_graph && python graph_rag.py && cd ..
cd module04_docling && python create_sample_pdf.py && python docling_rag.py
```